



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e  
INTERACCIÓN HUMANO COMPUTADORA



## **REPORTE DE PRÁCTICA N° 04**

**NOMBRE COMPLETO:** VELARDE VALENCIA JOSUE

**N° de Cuenta:** 320342330

**GRUPO DE LABORATORIO:** 13

**GRUPO DE TEORÍA:** 07

**SEMESTRE** 2027-1

**FECHA DE ENTREGA LÍMITE:** 23/09/2026

**CALIFICACIÓN:** \_\_\_\_\_

## REPORTE DE PRÁCTICA:

1. Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

Para el primer ejercicio retomé el modelo del Ejercicio de clase 4, corrigiendo el error que tuve de omitir el eje que gira las patas delanteras, por lo que continué modelando la pata del medio en +Z, incluyendo en el modelo una base cúbica azul que se une a un cilindro que funciona de eje para rotar hacia arriba la pata, que está unida a un prisma cuadrangular con 135 grados de rotación y en su extremo se encuentra el eje de la rueda. El eje se maneja con la tecla O y la rueda con la tecla I.

```
//BASE PARA LA LLANTA DE EN MEDIO +Z
// Base Azul
model = modelaux2;
model = glm::translate(model, glm::vec3(0.0f, -0.25f, 4.125f));
modelaux = model;
model = glm::scale(model, glm::vec3(1.0f, 1.0f, 0.25f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.0f, 1.0f); // Azul
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0] -> RenderMesh();
// Eje Gris
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.55f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion8()), glm::vec3(0.0f, 0.0f, 1.0f)); // O
modelaux = model;
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.25f, 1.0f, 0.25f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.3f, 0.3f, 0.3f); // Gris
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2] -> RenderMeshGeometry();
// Tubo Naranja
model = modelaux;
model = glm::translate(model, glm::vec3(-1.7f, -1.7f, 0.0f));
modelaux = model;
model = glm::rotate(model, glm::radians(135.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(0.5f, 4.5f, 0.8f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 0.5f, 0.0f); // Naranja
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0] -> RenderMeshGeometry();
//Eje Rueda Gris
model = modelaux;
model = glm::translate(model, glm::vec3(-1.3f, -1.3f, 1.6f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion9()), glm::vec3(0.0f, 0.0f, -1.0f)); // I
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.4f, 2.5f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.3f, 0.3f, 0.3f); // Gris
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2] -> RenderMeshGeometry();
// Rueda 3 Verde
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 0.2f, 0.0f));
model = glm::scale(model, glm::vec3(1.5f, 1.5f, 1.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.5f, 0.0f); // Verde
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2] -> RenderMeshGeometry();
```

La segunda pata del medio en -Z mantiene los mismos elementos de la pata de +Z, solo tuve que modificar los ejes en Z de la base, el eje de la pata, y el eje de la rueda. El eje funciona de igual manera con la tecla O, y la rueda gira con U.

```

//BASE PARA LA LLANTA DE EN MEDIO -Z
model = modelaux2;
model = glm::translate(model, glm::vec3(0.0f, -0.25f, -4.125f));
modelaux = model;
model = glm::scale(model, glm::vec3(1.0f, 1.0f, 0.25f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.0f, 1.0f); // Azul
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]-->RenderMesh();
// Eje Gris
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -0.55f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion8()), glm::vec3(0.0f, 0.0f, 1.0f)); // 0
modelaux = model;
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.25f, 1.0f, 0.25f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.3f, 0.3f, 0.3f); // Gris
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2]-->RenderMeshGeometry();
// Tubo Naranja
model = modelaux;
model = glm::translate(model, glm::vec3(-1.7f, -1.7f, 0.0f));
modelaux = model;
model = glm::rotate(model, glm::radians(135.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(0.5f, 4.5f, 0.8f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 0.5f, 0.0f); //Amarillo
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]-->RenderMeshGeometry();
//Eje Rueda Gris
model = modelaux;
model = glm::translate(model, glm::vec3(-1.3f, -1.3f, -1.6f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion10()), glm::vec3(0.0f, 0.0f, -1.0f)); // U
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.4f, 2.5f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.3f, 0.3f, 0.3f); //Gris
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2]-->RenderMeshGeometry();
// Rueda 3 Verde
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, -0.2f, 0.0f));
model = glm::scale(model, glm::vec3(1.5f, 1.5f, 1.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.5f, 0.0f); // Verde
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2]-->RenderMeshGeometry();

```

La rueda trasera en +Z mantiene también las figuras y estructura de las del medio, pero se encuentra 1 punto a la derecha y la rotación del tubo se cambió a -135 grados. El eje se mueve con la tecla de 0 y la rueda con la tecla 9.

```

//BASE PARA LA LLANTA DE ATRÁS +Z
// Base Azul
model = modelaux2;
model = glm::translate(model, glm::vec3(1.5f, -0.25f, 4.125f));
modelaux = model;
model = glm::scale(model, glm::vec3(1.0f, 1.0f, 0.25f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.0f, 1.0f); // Azul
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]-->RenderMesh();
// Eje Gris
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.55f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion11()), glm::vec3(0.0f, 0.0f, -1.0f)); // 8
modelaux = model;
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.25f, 1.0f, 0.25f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.3f, 0.3f, 0.3f); // Gris
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2]-->RenderMeshGeometry();
// Tubo Rojo
model = modelaux;
model = glm::translate(model, glm::vec3(1.7f, -1.7f, 0.0f));
modelaux = model;
model = glm::rotate(model, glm::radians(-135.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(0.5f, 4.5f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 0.0f, 0.0f); //Rojo
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[6]-->RenderMeshGeometry();
//Eje Rueda Gris
model = modelaux;
model = glm::translate(model, glm::vec3(1.3f, -1.3f, 1.6f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion12()), glm::vec3(0.0f, 0.0f, -1.0f)); //9
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.4f, 2.5f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.3f, 0.3f, 0.3f); //Gris
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2]-->RenderMeshGeometry();
// Rueda 3 Verde
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 0.2f, 0.0f));
model = glm::scale(model, glm::vec3(1.5f, 1.5f, 1.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.5f, 0.0f); // Verde
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2]-->RenderMeshGeometry();

```

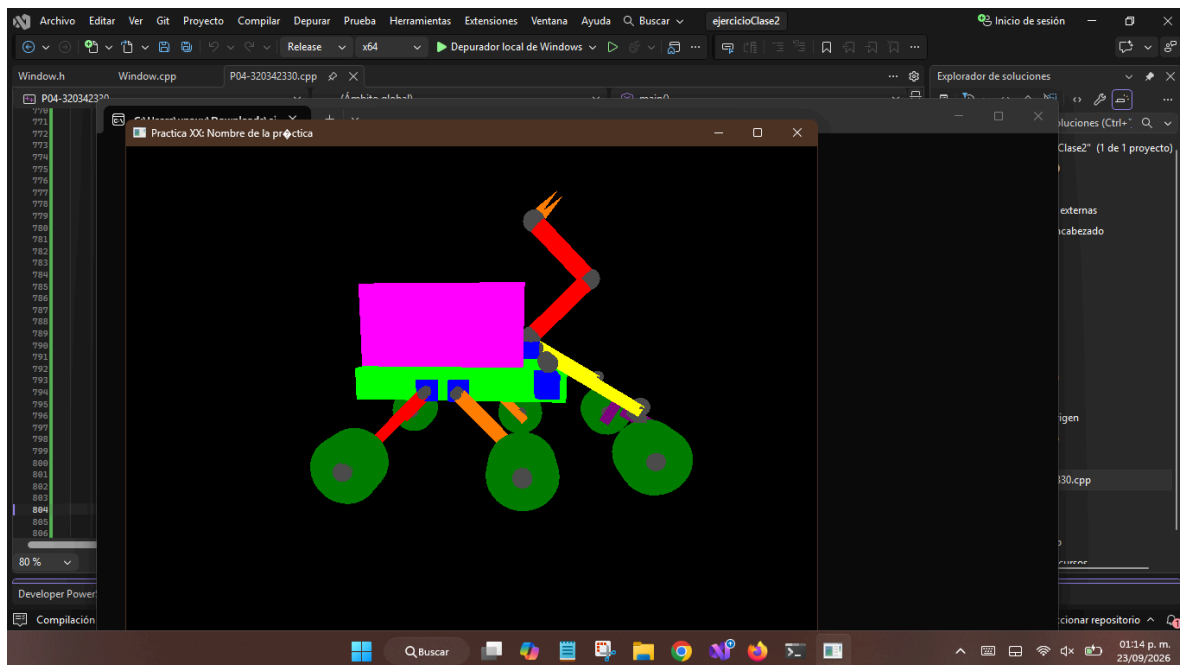
La última rueda en -Z es igual a la rueda trasera en +Z pero con coordenadas de z en negativo. El eje de igual forma se mueve con 0 y la rueda gira con la tecla 8.

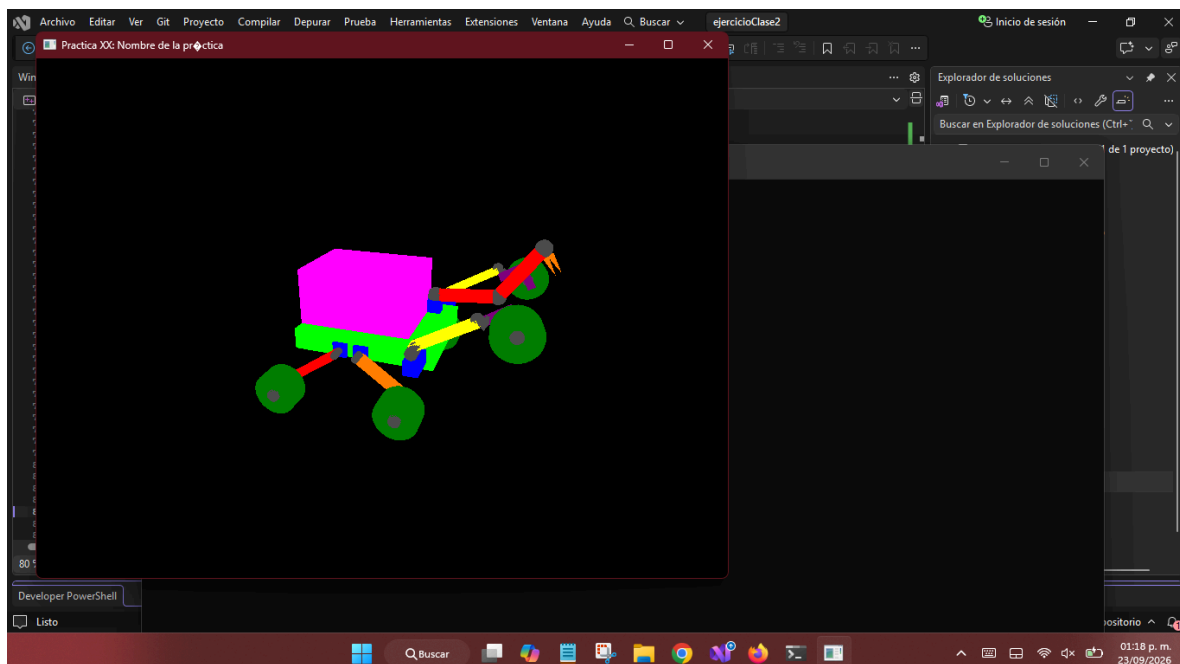
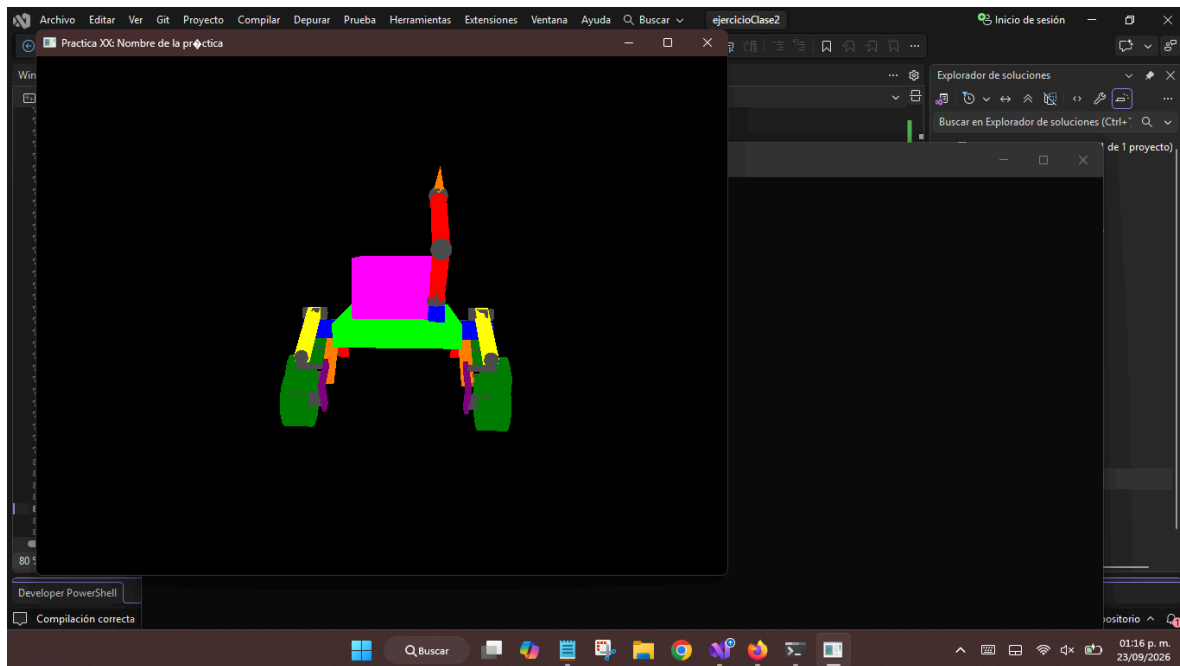
```

//BASE PARA LA LLANTA DE ATRAS -Z
// Base Azul
model = modelaux2;
model = glm::translate(model, glm::vec3(1.5f, -0.25f, -4.125f));
modelaux = model;
model = glm::scale(model, glm::vec3(1.0f, 1.0f, 0.25f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.0f, 1.0f); // Azul
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0] -> RenderMesh();
// Eje Gris
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -0.55f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion1()), glm::vec3(0.0f, 0.0f, -1.0f)); // 0
modelaux = model;
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.25f, 1.0f, 0.25f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.3f, 0.3f, 0.3f); // Gris
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2] -> RenderMeshGeometry();
// Tubo Rojo
model = modelaux;
model = glm::translate(model, glm::vec3(1.7f, -1.7f, 0.0f));
modelaux = model;
model = glm::rotate(model, glm::radians(-135.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(0.5f, 4.5f, 0.8f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 0.0f, 0.0f); // Rojo
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0] -> RenderMeshGeometry();
//Eje Rueda Gris
model = modelaux;
model = glm::translate(model, glm::vec3(1.3f, -1.3f, -1.6f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion3()), glm::vec3(0.0f, 0.0f, -1.0f)); // 8
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.4f, 2.5f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.3f, 0.3f, 0.3f); //Gris
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2] -> RenderMeshGeometry();
// Rueda 3 Verde
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, -0.2f, 0.0f));
model = glm::scale(model, glm::vec3(1.5f, 1.5f, 1.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.5f, 0.0f); // Verde
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2] -> RenderMeshGeometry();

```

La ejecución final se muestra a continuación:





Para el segundo ejercicio realicé el modelo de una sonda espacial, basándome de forma libre en la sonda DAWN.



Utilicé un cubo alargado café como base, otro cubo alargado para el soporte de los paneles de izquierda y derecha que también sirve de eje, 5 cubos planos para cada lado representando los paneles, un cilindro para representar el sensor frontal, un cilindro blanco pequeño como eje para la antena parabólica, un cono para la parabólica y una pirámide triangular para la aguja de la antena, y finalmente un cono para el propulsor inferior y una pirámide cuadrangular para simular el “plasma” que sale de la turbina.

```
// Limpiar la ventana
glClearColor(0.0f, 0.0f, 0.0f, 1.0f);
glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT); // Se agrega limpiar el buffer de profundidad
shaderList[0].useShader();
uniformModel = shaderList[0].getModelLocation();
uniformProjection = shaderList[0].getProjectLocation();
uniformView = shaderList[0].getViewLocation();
uniformColor = shaderList[0].getColorLocation();

// Sonda Espacial
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -10.0f));
modelaux = model;
modelaux2 = model;
// Base Café
model = modelaux2;
model = glm::scale(model, glm::vec3(5.0f, 7.0f, 5.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(0.478f, 0.255f, 0.067f); //Cafe
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMesh();

// Base Antena Blanca
model = modelaux2;
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 2.5f, 2.75f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion1()), glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion2()), glm::vec3(0.0f, -1.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.25f, 1.8f, 0.25f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f); //Blanco
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2]->RenderMeshGeometry();
```

```

// Antena Parabolica
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.5f));
model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(1.5f, 0.6f, 1.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.6f, 0.6f, 0.6f); //Gris
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[3]->RenderMeshGeometry();
// Antena
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 0.0f, 1.1f));
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.5f, 1.5f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f); //Blanco
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[1]->RenderMeshGeometry();

```

```

// Lado Derecho Gris
model = modelaux2;
model = glm::translate(model, glm::vec3(15.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion4()), glm::vec3(1.0f, 0.0f, 0.0f));
modelaux = model;
model = modelaux;
model = glm::scale(model, glm::vec3(25.0f, 1.0f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.6f, 0.6f, 0.6f); //Gris
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMeshGeometry();
//Paneles Azules
model = modelaux; //Primero
model = glm::translate(model, glm::vec3(12.5f, -0.2f, 0.35f));
modelaux = model;
model = glm::scale(model, glm::vec3(4.5f, 8.0f, 0.2f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.0f, 0.8f); // Azul
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMeshGeometry();
model = modelaux; //Segundo
model = glm::translate(model, glm::vec3(-5.0f, 0.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(4.5f, 8.0f, 0.2f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.0f, 0.8f); // Azul
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMeshGeometry();
model = modelaux; //Tercero
model = glm::translate(model, glm::vec3(-5.0f, 0.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(4.5f, 8.0f, 0.2f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.0f, 0.8f); // Azul
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMeshGeometry();

```

```

model = modelaux; //Cuarto
model = glm::translate(model, glm::vec3(-5.0f, 0.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(4.5f, 8.0f, 0.2f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.0f, 0.8f); // Azul
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0] -> RenderMeshGeometry();
model = modelaux; //Quinto
model = glm::translate(model, glm::vec3(-5.0f, 0.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(4.5f, 8.0f, 0.2f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.0f, 0.8f); // Azul
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0] -> RenderMeshGeometry();

```

```

// Lado Izquierdo Gris
model = modelaux2;
model = glm::translate(model, glm::vec3(-15.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion4()), glm::vec3(1.0f, 0.0f, 0.0f));
modelaux = model;
model = modelaux;
model = glm::scale(model, glm::vec3(25.0f, 1.0f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.6f, 0.6f, 0.6f); //Gris
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0] -> RenderMeshGeometry();
//Paneles Azules
model = modelaux; //Primero
model = glm::translate(model, glm::vec3(-12.5f, -0.2f, 0.35f));
modelaux = model;
model = glm::scale(model, glm::vec3(4.5f, 8.0f, 0.2f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.0f, 0.8f); //Azul
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0] -> RenderMeshGeometry();
model = modelaux; //Segundo
model = glm::translate(model, glm::vec3(5.0f, 0.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(4.5f, 8.0f, 0.2f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.0f, 0.8f); //Azul
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0] -> RenderMeshGeometry();
model = modelaux; //Tercero
model = glm::translate(model, glm::vec3(5.0f, 0.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(4.5f, 8.0f, 0.2f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.0f, 0.8f); //Azul
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0] -> RenderMeshGeometry();

```

```

model = modelaux; //Cuarto
model = glm::translate(model, glm::vec3(5.0f, 0.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(4.5f, 8.0f, 0.2f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.0f, 0.8f); //Azul
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]~>RenderMeshGeometry();
model = modelaux; //Quinto
model = glm::translate(model, glm::vec3(5.0f, 0.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(4.5f, 8.0f, 0.2f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.0f, 0.8f); //Azul
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]~>RenderMeshGeometry();

//Sensor delantero
model = modelaux2;
model = glm::translate(model, glm::vec3(0.0f, -3.0f, 3.0f));
modelaux = model;
model = modelaux;
model = glm::rotate(model, glm::radians(1020.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(1.5f, 0.5f, 1.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.4f, 0.4f, 0.4f); //Gris
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2]~>RenderMeshGeometry();

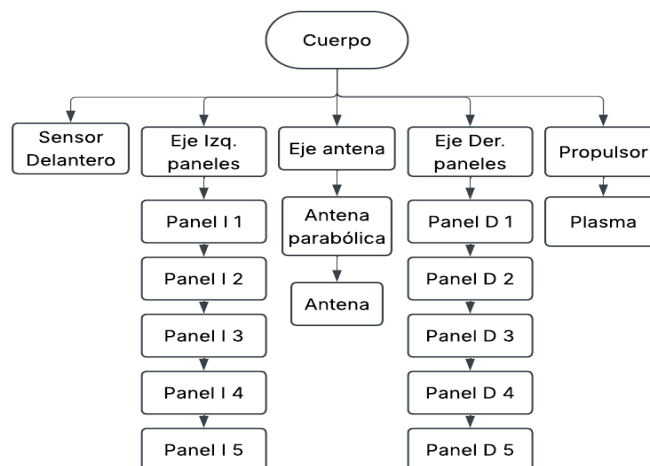
```

```

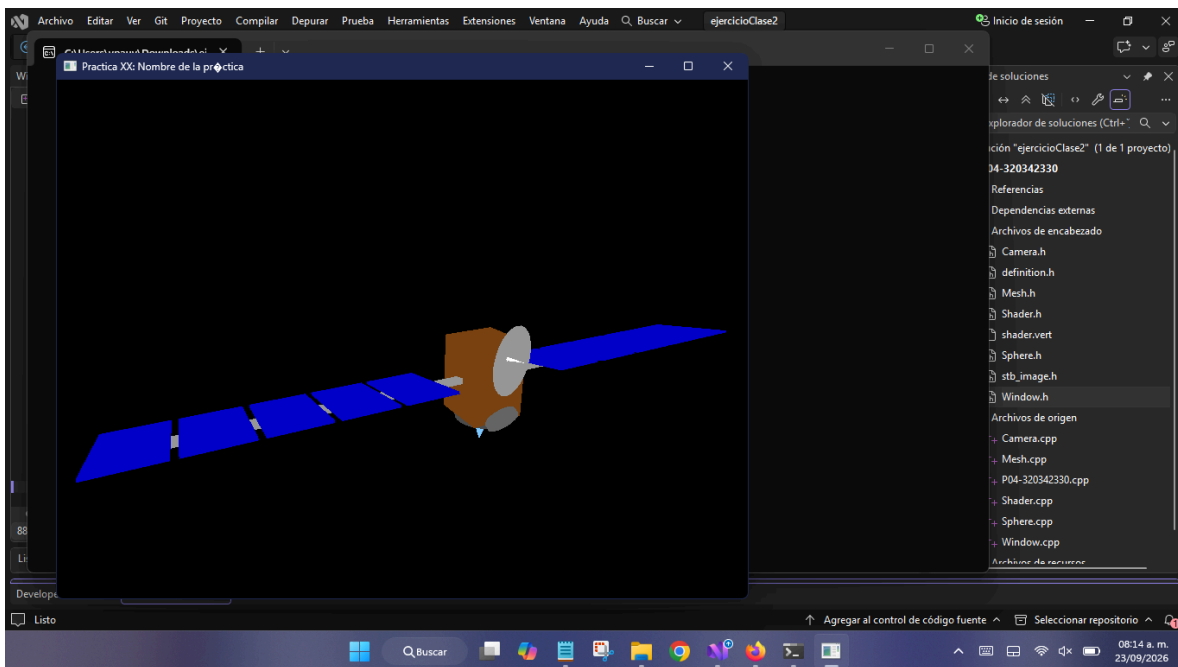
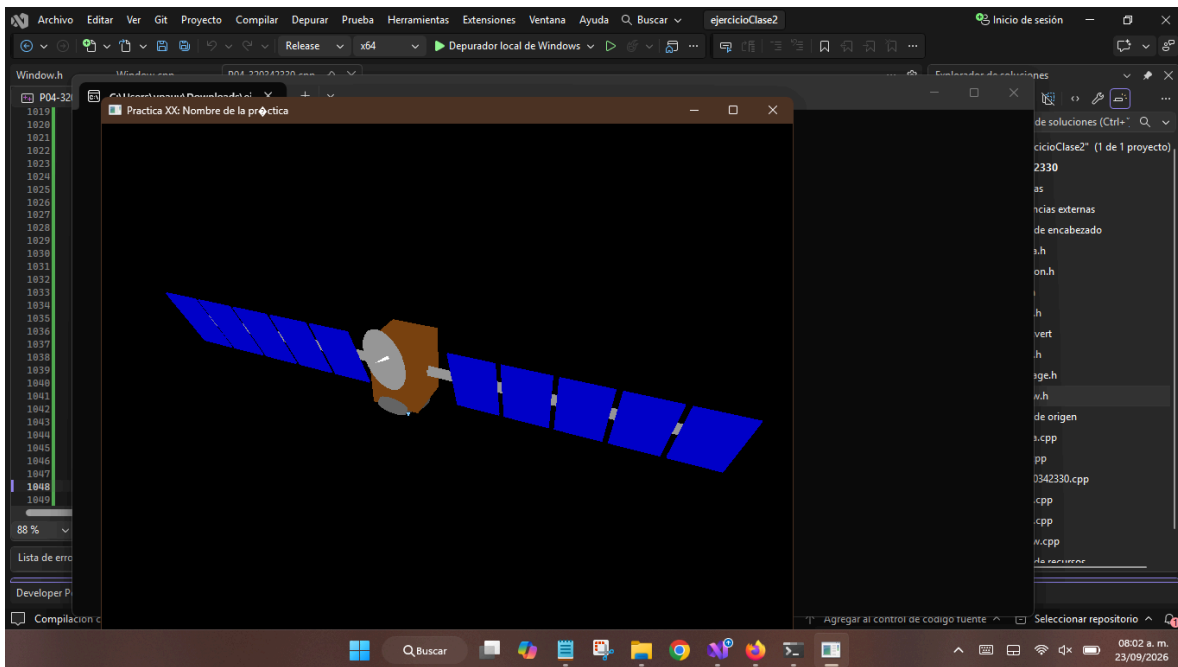
//Propulsor
model = modelaux2;
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, -0.5f, -3.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(1.2f, 1.2f, 1.2f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.3f, 0.3f, 0.3f); //Gris
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[3]~>RenderMeshGeometry();
// Plasma
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, -1.5f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion5()), glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, glm::radians(180.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(1.8f, 3.0f, 1.8f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.5f, 0.8f, 1.0f); //Azul Claro
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[4]~>RenderMeshGeometry();

```

La jerarquía es la siguiente:



El modelo final se ve de la siguiente manera:



2. Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

Presenté problemas con el manejo de las coordenadas y el uso de `modelaux`, `model` y `modelaux2`, porque había ocasiones en las que pasaba de un elemento a otro diferente pero no reiniciaba la posición del modelo al centro, sin embargo,

pude resolverlo poniendo más atención en las declaraciones y uso de model para que no ocurriera de nuevo.

### 3. Conclusión

Me pareció muy interesante desarrollar esta práctica, ya que pude comprender de mejor manera cómo utilizar distintas figuras geométricas para construir modelos más complejos, además de aplicar transformaciones como traslaciones, rotaciones y escalamiento para colocar correctamente cada elemento. También considero que el desarrollo de la sonda espacial me ayudó a entender mejor el uso de jerarquías y de variables como model, modelaux y modelaux2, pues fue importante controlar correctamente las coordenadas para evitar que las transformaciones de una figura afectaran a las demás. Los problemas que se presentaron durante la práctica me permitieron identificar mejor cómo reiniciar y organizar las transformaciones de cada elemento, logrando finalmente obtener los modelos esperados.